

Solution Report

Four people. One day. One connected sales workflow.

A product and architecture proposal for a sales platform that connects quotation, negotiation, approval, fulfillment, and billing.

Core decision	One full-stack TypeScript application and PostgreSQL.
Machine learning	No trained model required for the stated requirements.
Creative extension	Deal Lab: explain valid alternatives before changing a live deal.
Read with	The companion implementation plan and one-page architecture diagram.

Proposed design and implementation specification. Application code and deployment are future work; required features and optional innovations are distinguished throughout.

Contents

1. Executive decision	3
2. What the problem is asking for	3
3. People and their complete journeys	4
4. The product experience	4
5. Architecture and framework decisions	5
6. Required intelligence versus creative extensions	6
7. Do we need to build a machine-learning model?	6
8. Decisions the brief leaves open	7
9. What success looks like	8
10. Delivery strategy and risks	8
11. Architecture decision record	9
12. Sources and interpretation	9

Decision document • 5 September 2026 • Team: 4 • Build window: 24 hours

Product promise: A sales workspace that explains the consequences of a deal, governs its approval, and carries the accepted terms through stock allocation and billing.

Status: Proposed solution, not an implemented or benchmarked application. The companion Implementation Plan contains the build sequence, contracts, formulas, acceptance tests, and demo script.

1. Executive decision

Build DealFlow360 as one full-stack TypeScript application: Next.js 16, React, Tailwind CSS, shadcn/ui, Supabase Auth, and PostgreSQL. Keep authoritative business rules in server-side TypeScript modules and use database transactions for changes involving several records. Use one repository and one deployment.

Do not train a machine-learning model during the hackathon. Purchase-history statistics can drive real recommendations; explicit policies can drive discount governance, warehouse allocation, billing, and anomaly detection. The application can be intelligent without making unvalidated predictions.

The principal creative extension is **Deal Lab**: compare a small number of valid quotation alternatives and show the effect on customer price, margin, required approvals, and fulfillment. Build it only after the specified workflows pass. A unified explanation panel is the lower-cost alternative if time is tight.

Four people provide at most 96 gross person-hours. Coordination, integration, and rehearsal reduce productive build time. The full brief is ambitious; this plan makes that risk visible through checkpoints rather than assuming every feature will fit. Keep required business behavior and simplify presentation first.

2. What the problem is asking for

The brief describes a connected B2B quotation-to-cash system, with seven responsibilities:

Responsibility	Required result	Why it matters
Discount governance	Configurable customer/category ceilings; automatic Manager and Finance routing; audit history	Prevent unauthorized concessions
Recommendations	Ranked product suggestions, promotions, and immediate margin impact	Improve the deal while it is being built
Fulfillment	Stock-aware warehouse split, override, backorder, and consolidation after receipt	Make delivery decisions reflect inventory
Hybrid billing	One-time items and recurring plans, schedules, proration, cancellation, credits	Preserve the commercial agreement through billing
Customer negotiation	Separate restricted portal, line comments, change requests, counter-discounts, confirmation	Collaborate on the actual quotation
Deal monitoring	Stalled quotes, unusual discounts, promise slippage, nudges/escalations	Surface problems early
Configuration/reporting	Working setup forms, filters, PDF and spreadsheet exports	Make behavior configurable and results inspectable

The brief explicitly permits any language, framework, or database. Building an Odoo module is not required by this PDF. It explicitly rejects fake approval, inventory, and billing logic. The customer portal must enforce restricted access on the server. [Brief, pp. 2–10]

The optional wording applies to A6, the recommendation rule-configuration screen. The actual recommendation panel is required by B5 and the quick test. Multi-company and multi-currency execution are bonuses. [Brief, pp. 5, 7, 10–11]

3. People and their complete journeys

Sales representative

Sign up/log in → select customer → build a quotation → see live totals, margin, risk reasons, and suggestions → add/dismiss a suggestion → submit for required approval → share the customer portal link → respond to negotiation → follow fulfillment and billing.

Manager and Finance/Operations

Manager reviews the exact submitted quote version and the policy reasons. They approve, reject, or return it for revision. Finance acts after Manager when required. Operations reviews warehouse allocation and backorders; Finance records payments and handles credits/refund records. Actions are attributed and time-stamped.

Customer

Log into the portal → see only their quotations → comment on a particular line or propose revised commercial terms → see whether approval is pending → accept the exact terms displayed. An acceptance with outstanding approvals is conditional and cannot release stock or create a billable order.

Administrator

Configure products/variants, prices, customer tiers, category ceilings, approval thresholds, warehouses, stock, replenishment thresholds, shipping weights, plans, proration/cancellation rules, and alert thresholds. Configuration changes affect future evaluations; submitted versions preserve their applicable policy snapshot.

4. The product experience

Use a desktop-first workspace that fits a laptop projector. A quotation has one detail page with tabs for **Build, Approvals, Customer Activity, Fulfillment, Billing, and History**. Avoid forcing users through unrelated pages to understand one deal.

The builder has three regions: searchable catalog, editable quote lines, and a context panel. The context panel shows required approval with reasons, recommendation cards, and fulfillment availability. Display the amount due initially separately from recurring commitments. Label recurring margin calculations with their billing period.

The portal uses its own layout and API responses. It displays customer prices and terms, without internal costs, margins, risk thresholds, other customers, or unrelated inventory detail. Provide a clear version label and pending-approval banner.

The manager dashboard prioritizes actionable deals. Each alert opens the relevant quotation and a real action. Use consistent colors with written labels: green for ready, amber for waiting/risk, red for blocked. Charts must link to live filtered records; decorative metrics add little value.

5. Architecture and framework decisions

Layer	Selected approach	Reason
Application	Next.js 16 App Router, TypeScript, React	One language, one deployable, shared contracts
UI	Tailwind CSS, shadcn/ui, Lucide icons	Reusable forms, tables, dialogs, and accessible controls
Forms/data	React Hook Form, Zod, TanStack Query	Shared validation, controlled edits, refresh/invalidation
Identity	Supabase Auth, email/password, SSR integration	Avoid building password/session infrastructure
Data	Supabase PostgreSQL, SQL migrations, node-postgres (pg)	Relational constraints, explicit transactions and locks
Money	Integer minor units in PostgreSQL; decimal.js for intermediate arithmetic	Predictable rounding and exact persisted amounts
Reports	Recharts, pdf-lib, ExcelJS	Live charts plus downloadable PDF/XLSX
Tests	Vitest and Playwright	Formula, concurrency, permission, and end-to-end checks
Hosting/jobs	Vercel application; Supabase database/Auth/Cron	Managed hosting, scheduled billing and alerts

Use the latest patched compatible release within the chosen major at setup and commit the lockfile. Use Node.js 24 LTS as the project target, subject to the host's supported runtime at setup. The verified Next.js documentation requires at least Node 20.9; that minimum is not a recommendation to use an obsolete runtime. [Next.js installation](#), [Node.js releases](#)

Keep database access server-side. Use Supabase for authentication, not as a second independent business API in the browser. The documented SSR integration supports verified identity and cookie refresh; authorization still comes from the application's stored roles and record ownership. [Supabase SSR](#)

A modular monolith is the smallest architecture that lets four developers work on separate domains while retaining atomic order creation, stock reservation, and billing. Microservices would introduce deployment and consistency work that the user does not benefit from during this event.

6. Required intelligence versus creative extensions

Capability	Already in the brief?	Proposed implementation
Upsell/cross-sell	Yes	Purchase-history association counts plus promotion and margin rules
Discount approval	Yes	Explainable, configurable risk calculation and sequential approvals
Warehouse optimization	Yes	Deterministic search over a small warehouse set
Anomaly/stalled alerts	Yes	Statistical comparisons and explicit thresholds
Customer counteroffer	Yes	Versioned proposal with automatic re-evaluation
Deal Lab alternatives	No	Counterfactual alternatives evaluated by the same real engines
Unified decision explanation	Extends required audit/risk views	Connect pricing, approval, inventory, and billing reasons in one readable view

Do not describe baseline features as extra innovation. Do not claim worldwide novelty without evidence. The distinctive contribution is the integration, clarity, and quality of execution.

Signature extension: Deal Lab

For a quote awaiting approval, generate at most three candidate alternatives from the actual catalog and policy:

1. **Approval-ready:** reduce excessive discounts to configured safe values, and recheck both aggregate policy and margin constraints.
2. **Margin-focused:** add or substitute an eligible product only when the resulting customer price and margin meet stated constraints.
3. **Delivery-focused:** use an explicitly configured equivalent product with available stock, displaying any price or specification change.

Each candidate shows the change from the current quote: price, margin amount/percentage, approval chain, shipment estimate, and unfilled quantity. Report when no feasible alternative exists. A rep applies one explicitly; the server revalidates current stock, prices, policy, and quote version before saving a new revision. A scenario never reserves inventory or alters an approved deal by itself.

Scope control: implement approval-ready first. The other two are stretch features. Label any heuristic result as a suggested alternative, not a provably optimal deal.

Lower-cost extension: one decision explanation

Example: “Finance is needed because the service discount exceeds its category ceiling by 8 percentage points. One laptop is backordered. This order includes INR 39,300 once and INR 3,000 per month before tax.”

Every sentence comes from structured engine outputs. Links open the triggering line, approval step, stock shortage, or billing schedule. It requires no language model and gives judges a clear explanation of the system's reasoning.

7. Do we need to build a machine-learning model?

No. No requirement in this PDF forces model training.

Problem	Suitable 24-hour technique	Why training is unnecessary
Product recommendations	Co-purchase confidence with promotion boost and margin filter	Uses actual order history directly
Discount anomaly	Historical mean/standard deviation plus absolute difference threshold	Transparent, testable, works with small demo history
Approval	Policy formula and state machine	This is a business constraint, not a prediction
Warehouse allocation	Small combinatorial search	This is an optimization problem
Proration	Calendar-based arithmetic	Billing needs deterministic amounts
Deal Lab	Generate a few candidates and evaluate them	Reuses the existing engines

Use representative synthetic history, clearly labeled as demo data. It must contain real stored order lines that the recommendation query aggregates; do not return fixed recommendation cards. When history is insufficient, return eligible promoted/category alternatives with a “limited history” reason, or no suggestion.

An optional LLM could paraphrase a structured deal explanation or draft a negotiation response for a human to review. It must not decide prices, approve deals, reserve stock, issue invoices, or send messages. It is not needed for the planned demo and is excluded from the 24-hour critical path.

After the hackathon, a learned recommendation/ranking model could be evaluated on time-separated real interaction data. Compare it with the rules baseline using ranking quality and business outcomes; do not claim accuracy or conversion lift from synthetic data. Predictive win probability needs reliable outcome labels, sufficient history, and calibration before it is useful.

8. Decisions the brief leaves open

Ambiguity	Proposed explicit decision
Exact blended-risk formula	Evaluate worst line, value-weighted excess, total excess value, and overall discount/margin limits; route to the highest level
Approval versus customer confirmation order	Both are independent gates; create an executable order only when the current version meets both
Stock before customer acceptance	Preview only; reserve transactionally when the order becomes executable
Meaning of hybrid total	Show one-time amount, each recurring plan/cadence, and initial amount due separately
Proration convention	Calendar-day fraction with start-inclusive/end-exclusive periods
Portal counteroffer	Create a candidate revision, immediately route its risk, require seller adoption and customer acceptance before execution
Real-time behavior	Refresh immediately after actions and poll every five seconds; display last refresh time
Payment scope	Persist payment/credit/refund records; a real payment gateway is a future integration
Multi-currency	Currency-aware price records; execute the hackathon in INR with no FX conversion
“XLS” export	Provide real XLSX; verify whether the organizer literally requires legacy XLS

These are team design choices, not formulas supplied by the organizers. Record them in the README and make the behavior visible in the demo. The numerical defaults are illustrative commercial policies, not legal or financial rules.

9. What success looks like

The required deliverable is a working application, seeded data, a five-minute live demo with two complete journeys, a one-page architecture/data-model diagram, and a short roadmap. [Brief, p. 10]

The demo should prove:

- A normal quote completes without unnecessary approval, accepts a recommendation, reserves/dispatches stock, creates an invoice, and becomes paid after a recorded receipt.
- A mixed hardware/service/subscription quote triggers sequential approval, receives a customer counteroffer, invalidates earlier approvals, clears the new gates, splits across warehouses, and handles a real backorder receipt.
- Billing separates one-time and recurring charges; a configured mid-cycle quantity change creates a calculated adjustment; cancellation follows the configured credit policy.
- Another customer's quotation is inaccessible, duplicate actions do not duplicate financial records, and stock does not become negative.
- Reporting and alerts reflect persisted activity, not hardcoded dashboards.

The five-minute script cannot show every edge case. Automated acceptance evidence covers the rest. Performance goals are targets, not measured claims: typical quote evaluation below 500 ms on the demo dataset; user-visible freshness within five seconds; zero duplicate orders/invoices in retry tests.

10. Delivery strategy and risks

Members own vertical modules: A owns platform/quote/governance; B owns fulfillment; C owns recurring billing/payments; D owns portal/recommendations/reporting. Each builds their corresponding server code and screens using shared components. Merge small working changes every two hours. All four agree on database keys, money units, statuses, and service signatures at the start.

The first integrated ordinary quote should work by hour 8. Both ordinary and negotiated mixed flows should work by hour 16. Required edge cases, exports, and alerts should pass by hour 20. Innovation gets a strictly bounded slot after that; the final hours are for fixes and rehearsal.

Risk	Containment
Four developers create incompatible modules	Shared TypeScript contracts and centrally reviewed migrations in the first hour
Quote edits accidentally retain approval	Immutable submitted versions; version-specific decisions and acceptances
Concurrent confirmations oversell stock	Row locks and atomic reservation with nonnegative constraints
Recurring jobs create duplicate invoices	Unique charge keys, job locks, and retry-safe writes
Portal reveals internal data	Separate DTOs, ownership checks, and negative access tests
Configuration is only cosmetic	Tests change a setting and verify the next evaluation changes
Deployment fails near the deadline	Deploy login and one database query by hour 2; keep a tested local app fallback

Risk	Containment
Innovation consumes completion time	Cut Deal Lab variants/LLM before cutting baseline requirements

11. Architecture decision record

Context: Four developers, one day, many connected transactional business rules, no confirmed existing codebase or training dataset.

Decision: One Next.js application, one PostgreSQL database, managed authentication, explicit domain services, integer persisted money, versioned quotations, and deterministic decision engines.

Consequences: Fast local reasoning and shared types; coordinated schema changes and careful server authorization are essential. Domain modules remain separable if independent scaling is needed later.

Evolution triggers: Move jobs to a dedicated worker when job duration/retries exceed the deployment budget. Add a learned ranker when real history supports evaluation. Add FX, multi-company, gateway, and carrier integrations only when required by actual usage.

12. Sources and interpretation

Primary requirements: **DealFlow360.pdf**, 13 pages, provided by the team at `C:/Users/Nikhil1616/Downloads/DealFlow360.pdf`. References to page numbers above refer to that PDF. The linked Excalidraw mockup was not retrievable, so no unobserved mockup behavior is assumed.

The PDF is source material for the product requirements, not an instruction to execute its login, payment, or external-message walkthrough now. This deliverable is a report and implementation plan only.

Technical references checked on 5 September 2026: [Next.js](#), [Supabase SSR](#), [PostgreSQL connections](#), [node-postgres transactions](#), [PostgreSQL locking](#), [Supabase Cron](#), [shadcn/ui](#), [pdf-lib](#). Stack selection, risk formulas, UX, effort allocation, and innovation concepts are this report's proposed design judgments.